Mit welchen Methoden ermittle ich Keywords aus einem Text / Artikel?

Was ist die aktuelle Best Practise?
Wir kann ich das durch lokale LLM stützen?

Now I have comprehensive research. Let me generate the final expert report in German:

Keyword-Extraktion aus Texten: Methoden, Best Practices und lokale LLM-Unterstützung

Executive Summary

Keyword-Extraktion ist eine fundamentale Aufgabe im Natural Language Processing (NLP), die automatisch die wichtigsten Begriffe und Phrasen aus Dokumenten identifiziert. Diese Analyse untersucht den aktuellen Stand der Technik (Stand Januar 2026) und präsentiert eine systematische Bewertung von statistischen, graph-basierten, embedding-basierten und LLM-gestützten Methoden. Die Forschungsergebnisse zeigen, dass hybride Ansätze, die mehrere Techniken kombinieren, die höchste Genauigkeit erreichen, während lokale Large Language Models (LLMs) zunehmend praktikable Alternativen zu Cloud-basierten Lösungen bieten.[1] [2] [3] [4]

Methodologische Taxonomie der Keyword-Extraktion

Die Keyword-Extraktion lässt sich in vier primäre Methodenkategorien unterteilen, die jeweils unterschiedliche algorithmische Paradigmen repräsentieren und spezifische Anwendungsfälle adressieren.

Vergleich verschiedener Keyword-Extraktionsmethoden nach Typ, Geschwindigkeit, Genauigkeit und Hauptvorteilen

Statistische Methoden

Statistische Verfahren bilden die Grundlage der Keyword-Extraktion und zeichnen sich durch hohe Geschwindigkeit sowie Domänenunabhängigkeit aus. Diese Methoden operieren primär auf Frequenz- und Distributionsmerkmalen.

TF-IDF (Term Frequency-Inverse Document Frequency) multipliziert die Termfrequenz in einem Dokument mit der inversen Dokumentfrequenz im Korpus. Diese Methode identifiziert Wörter, die in einem spezifischen Dokument häufig vorkommen, jedoch im Gesamtkorpus selten sind. TF-IDF erreicht Genauigkeiten von bis zu 88% bei Textklassifikationsaufgaben und bietet eine präzise

Baseline mit 0.90 Precision für kategoriale Unterscheidungen. Die Hauptlimitation besteht in der Unfähigkeit, semantische Beziehungen oder kontextuelle Bedeutungen zu erfassen – ein Wort, das nur einmal vorkommt, aber inhaltlich zentral ist, wird systematisch unterbewertet.[1] [5] [6] [7]

YAKE (Yet Another Keyword Extractor) überwindet die Korpus-Abhängigkeit von TF-IDF durch Analyse von fünf statistischen Features innerhalb eines einzelnen Dokuments: Termpositionen (Begriffe am Textanfang erhalten höhere Gewichtung), normalisierte Termfrequenz, Kontextrelationalität (Ko-Okkurrenz mit diversen Begriffen signalisiert geringere Bedeutung), Satzverteilung und Levenshtein-basierte Deduplizierung. YAKE funktioniert sprachunabhängig und erzielt F1-Scores zwischen 74-81%, wobei die Performance bei kurzen Texten besonders robust ist.[6] [7] [8] [1]

RAKE (Rapid Automatic Keyword Extraction) extrahiert Keywords durch Identifikation von Ko-Okkurrenz-Mustern und Worthäufigkeiten. Die Methode segmentiert Text anhand von Stoppwörtern und Interpunktion, bewertet Kandidaten basierend auf Frequenz und Nähe zu anderen Begriffen, und rankt sie entsprechend. RAKE verarbeitet 2.000 Dokumente in zwei Sekunden und erreicht damit die höchste Zeiteffizienz aller Methoden. Die algorithmische Einfachheit resultiert jedoch in begrenztem semantischem Verständnis.[5] [8]

Graph-basierte Methoden

Graph-basierte Ansätze modellieren Texte als Netzwerke, in denen Knoten Terme repräsentieren und Kanten Ko-Okkurrenz-Beziehungen abbilden. Diese Methoden nutzen Zentralitätsmaße zur Identifikation wichtiger Keywords.

TextRank adaptiert den PageRank-Algorithmus für Textanalyse. Der Algorithmus konstruiert einen gerichteten Graphen, in dem Wörter mit spezifischen Part-of-Speech-Tags (typischerweise Nomen und Adjektive) als Knoten fungieren. Kanten werden zwischen Begriffen erstellt, die innerhalb eines definierten Kontextfensters ko-okkurrieren, mit Gewichtungen basierend auf Frequenz. Durch iterative Berechnung über 20-30 Iterationen bis zur Konvergenz (Threshold: 0.0001) stabilisieren sich die Vertex-Gewichte. Die Top 33% der höchstbewerteten Knoten werden als Keywords extrahiert, wobei konsekutive Begriffe zu Multi-Word-Keyphrases kombiniert werden. TextRank erreicht F1-Scores um 75% und identifiziert effektiv lokal wichtige Terme.[1] [5] [9] [10] [11]

Performance-Vergleich verschiedener Keyword-Extraktionsmethoden nach F1@10 Score (höhere Werte = bessere Performance)

TopicRank erweitert TextRank durch thematische Clusterbildung. Statt einzelne Begriffe zu ranken, gruppiert TopicRank semantisch ähnliche Keyword-Kandidaten in Topic-Cluster und extrahiert anschließend die repräsentativsten Keywords aus den wichtigsten Topics. Diese Methode erfasst dokumentübergreifende Themenzusammenhänge effektiver als TextRank, erfordert jedoch zusätzliche Rechenressourcen für das Clustering.[1]

TRISUM repräsentiert einen innovativen Hybrid-Ansatz, der TextRank (lokale Wichtigkeit), Eigenvector Centrality (globale Einflussnahme) und Betweenness Centrality (Brückenbegriffe zwischen Themen) kombiniert. Die Weighted Ensemble Strategy aggregiert Scores aus allen drei Methoden und boosted Terme, die von mehreren Algorithmen identifiziert werden. Evaluationen

zeigen, dass TRISUM die Biases einzelner Methoden reduziert und sowohl hochfrequente lokale Terme als auch global signifikante Konzepte erfasst.[3]

Typische Pipeline für Keyword-Extraktion mit vier Hauptphasen: Preprocessing, Kandidatengenerierung, Scoring und Post-processing

Embedding-basierte Methoden

Embedding-basierte Verfahren nutzen neuronale Sprachmodelle zur Generierung kontextueller Wortrepräsentationen, die semantische Ähnlichkeiten quantifizierbar machen.

KeyBERT verwendet BERT-basierte Sentence Transformers zur Vektorisierung sowohl des Gesamtdokuments als auch individueller Keyword-Kandidaten. Die Methode berechnet Cosine Similarity zwischen Dokument-Embedding und Kandidaten-Embeddings – Keywords mit höchster Ähnlichkeit zum Dokument werden als semantisch zentral klassifiziert. KeyBERT unterstützt diversifizierungsstrategien wie **Maximal Marginal Relevance (MMR)** und **Max Sum Similarity**. MMR balanciert Dokumentrelevanz gegen Keyword-Diversität durch die Formel: `MMR = (1-λ) × Ähnlichkeit(Kandidat, Dokument) - λ × max(Ähnlichkeit(Kandidat, ausgewählte Keywords))`, wobei λ den Diversitätsparameter (0-1) darstellt. Evaluationen zeigen F1@10-Scores von 82% und eine durchschnittliche Extraktionszeit von 184 Sekunden für mittlere Dokumentsammlungen.[12] [13] [14] [15] [16]

Performance-Vergleich verschiedener Keyword-Extraktionsmethoden nach F1@10 Score (höhere Werte = bessere Performance)

SIFRank fusioniert Sentence-in-Files (SIF)-Embeddings mit ELMo-basierten kontextuellen Repräsentationen. Die Methode integriert positionsbasierte Gewichtung (Position-biased Weighting) zur Priorisierung von Begriffen am Dokumentanfang und erreicht besonders bei kompakten Dokumenten robuste Performance mit F1-Scores um 81%.[17]

Fine-tuned Transformer-Modelle repräsentieren den aktuellen State-of-the-Art für supervised Keyword-Extraktion. DeBERTa mit LoRA (Low-Rank Adaptation) erreicht 88% F1@10 durch selektive Finetuning von Query- und Value-Matrizen bei gefrorenen Backbone-Layern. Die Integration von Mixture-of-Experts (MoE)-Schichten ermöglicht Spezialisierung auf unterschiedliche syntaktische und semantische Strukturen – Experten fokussieren auf Interpunktion, Part-of-Speech-Tags, Named Entities oder domänenspezifische Terminologie. BiLSTM-Klassifikationsköpfe verbessern die Multi-Word-Keyphrase-Extraktion signifikant, indem sie sequenzielle Abhängigkeiten zwischen Token modellieren.[18] [19] [20]

LLM-basierte Methoden

Large Language Models bieten die höchste semantische Verständnistiefe, jedoch mit erhöhten Rechenanforderungen und Latenz.

KeyLLM erweitert KeyBERT um LLM-gestützte Verfeinerung. Der Workflow umfasst drei Betriebsmodi: (1) **Zero-Shot Extraction** – direktes Prompting des LLMs für Keyword-Generierung, (2) **Efficient Clustering** – Gruppierung semantisch ähnlicher Dokumente mit anschließender Single-LLM-Inferenz pro Cluster (threshold-basiert), und (3) **KeyBERT+KeyLLM Fusion** – KeyBERT generiert Kandidaten, die vom LLM kontextuell evaluiert und gefiltert werden.

Implementierungen mit Mistral 7B erreichen vergleichbare Qualität zu GPT-4 bei lokaler Ausführung, wobei GGUF-Quantisierung die Inferenzgeschwindigkeit auf Consumer-Hardware ermöglicht.[12] [2] [21] [22] [23]

MAPEX (Multi-Agent Pipeline for Keyphrase Extraction) koordiniert drei spezialisierte LLM-Agenten durch eine adaptive Dual-Path-Strategie. Der **Expert Recruiter** analysiert Dokumentcharakteristika und weist domänenspezifische Rollen zu (z.B. „Informatik-Experte", „Biomedizin-Spezialist"). Der **Candidate Extractor** generiert einen breiten Pool lexikalischer Variationen. Für kurze Dokumente (< 300 Wörter) aktiviert MAPEX den **Knowledge-Driven Path**: Kandidaten werden mit Wikipedia-Definitionen angereichert, und der Domain Expert rerankt basierend auf semantischer Kontextualität. Für lange Dokumente nutzt der **Topic-Guided Path** thematische Extraktion zur Reduktion von „Semantic Dilution" bei großen Kontextfenstern. Benchmarks zeigen 2.44% F1-Verbesserung gegenüber SOTA-unsupervised-Methoden und 4.01% über Standard-LLM-Baselines.[4]

Performance-Vergleich verschiedener Keyword-Extraktionsmethoden nach F1@10 Score (höhere Werte = bessere Performance)

Proprietäre Cloud-LLMs (GPT-4o-mini, Gemini-1.5-flash-8B) übertreffen traditionelle unsupervised Methoden konsistent, mit F1@10-Scores von 85-86%. Strukturierte Ausgabeformate (JSON-Schemas) ermöglichen deterministische Keyword-Extraktion mit Metadaten (Relevanzscores, Character-Offsets). Die Kostenstruktur (typisch $0.10-0.50 per 1M Input-Tokens) macht Cloud-LLMs für Produktionssysteme bei hohem Volumen ökonomisch herausfordernd.[24] [25] [18]

Best Practices für Produktionssysteme

Die Implementierung produktionsreifer Keyword-Extraktion erfordert eine systematische Pipeline-Architektur, die Preprocessing, Kandidatengenerierung, Scoring und Post-Processing integriert.

Pipeline-Architektur

Phase 1: Text Preprocessing umfasst Tokenisierung (sentence- und word-level), Stopword-Entfernung (domänenspezifische Stoplisten), Part-of-Speech-Tagging und Lemmatisierung. Für deutsche Texte empfiehlt sich RFTagger für POS-Annotation und Stanford CoreNLP für dependency parsing. Preprocessing reduziert Rauschen und standardisiert linguistische Variationen – ein kritischer Schritt, der die Downstream-Performance um 10-15% verbessern kann.[26] [11] [27] [28] [29] [30]

Phase 2: Kandidatengenerierung extrahiert potenzielle Keywords durch N-gram-Extraktion (Unigrams bis Trigrams), Noun-Chunk-Parsing, Named Entity Recognition und POS-basierte Pattern-Matching. CountVectorizer mit `ngram_range=(1,3)` und domänenspezifischen Stoplisten bietet eine effiziente Baseline-Implementierung. Fortgeschrittene Ansätze nutzen spaCy's Noun Chunks für Multi-Word-Expressions:[31] [32] [30] [26]

Typische Pipeline für Keyword-Extraktion mit vier Hauptphasen: Preprocessing, Kandidatengenerierung, Scoring und Post-processing

```
nlp = spacy.load("de_core_news_lg")
doc = nlp(text)
candidates = [chunk.text for chunk in doc.noun_chunks
              if chunk.text.lower() not in stopwords]
```

Phase 3: Feature-Extraktion und Scoring kombiniert statistische (TF-IDF, Frequenz), graph-basierte (Zentralität), semantische (Embedding-Ähnlichkeit) und LLM-basierte Features. Hybride Scoring-Funktionen aggregieren multiple Signale durch gewichtete Summen oder Ensemble-Learning. Die Kontextualisierung durch Embeddings verbessert die semantische Präzision signifikant – Evaluationen zeigen, dass kontextbasierte Methoden frequenzbasierte Ansätze um 15-25% übertreffen.[3] [33]

Phase 4: Post-Processing implementiert Ranking, Deduplizierung (Levenshtein-Distance < 2 für Near-Duplicates), Diversitätsfilterung (MMR mit λ=0.5-0.7) und Top-K-Selektion. Post-Processing-Optimierungen, die PoS-Tagging und semantische Anreicherung integrieren, steigern F1-Scores um durchschnittlich 25.8% über alle Methoden hinweg.[6] [14] [15] [28]

Entscheidungsbaum zur Auswahl der geeigneten Keyword-Extraktionsmethode basierend auf verfügbaren Ressourcen und Anforderungen

Methodenwahl: Entscheidungskriterien

Die optimale Methodenwahl basiert auf vier Dimensionen: Datencharakteristika, Ressourcenverfügbarkeit, Performance-Anforderungen und Latenztoleranz.

Mit gelabelten Trainingsdaten dominieren Fine-tuned Transformer-Modelle. DeBERTa mit LoRA erreicht SOTA-Performance (88% F1@10) bei moderaten Trainingsanforderungen – 4-Bit-GPTQ-Quantisierung reduziert Speicherbedarf auf ~2GB GPU-RAM. Validation-Set-Training (20 Epochen, Early Stopping) auf 1.000-5.000 Dokumenten genügt für robuste Generalisierung.[34] [18]

Performance-Vergleich verschiedener Keyword-Extraktionsmethoden nach F1@10 Score (höhere Werte = bessere Performance)

Ohne Trainingsdaten bei **verfügbarer GPU**: KeyBERT mit `all-MiniLM-L6-v2` bietet optimales Geschwindigkeit-Qualität-Verhältnis (F1: 82%, 184s für mittlere Korpora). KeyLLM mit Mistral 7B steigert Qualität weiter (F1: ~84%), erfordert jedoch 12-16GB VRAM.[23] [35] [15] [16]

CPU-Only-Szenarien erfordern Kompromisse zwischen Geschwindigkeit und Qualität. RAKE verarbeitet 2.000 Dokumente in 2 Sekunden und bietet solide Baseline-Performance (F1: 76%). YAKE verbessert Qualität leicht (F1: 74%) bei sprachunabhängiger Anwendbarkeit. Für höchste CPU-Performance empfiehlt sich TF-IDF mit preprocessing-optimierter scikit-learn-Pipeline (F1: 78%, <5s für 500 Dokumente).[1] [7] [8] [30]

Hybrid-Ansätze kombinieren multiple Methoden zur Reduktion methodenspezifischer Biases. Eine praktische Implementierung: YAKE für schnelle Kandidatengenerierung → KeyBERT für semantisches Reranking → LLM für finale Filterung bei ambivalenten Fällen. Dieser Ansatz balanciert Effizienz gegen Präzision und erreicht F1-Scores von 85-87%.[3] [4] [36]

Lokale LLM-Integration: Praktische Implementierung

Die lokale Ausführung von LLMs eliminiert Cloud-Abhängigkeiten, reduziert Laufzeitkosten und gewährleistet Datensouveränität – kritische Faktoren für Enterprise-Anwendungen.

Modellselektion für Keyword-Extraktion

Llama 3.2 (1B/3B) bietet minimale Ressourcenanforderungen bei akzeptabler Qualität für strukturierte Extraktion. Die 1B-Variante läuft auf CPUs mit 4-8GB RAM, während 3B moderate GPU-Beschleunigung (4GB VRAM) unterstützt. Quantisierung auf Q4 reduziert Modellgröße auf 600MB-2GB bei <5% Performance-Verlust.[37]

Mistral 7B repräsentiert den Sweet Spot für Keyword-Extraktion – 7 Milliarden Parameter balancieren semantisches Verständnis gegen Inferenzgeschwindigkeit. KeyLLM-Implementierungen mit Mistral 7B erreichen vergleichbare Qualität zu GPT-4 bei 5-10x niedrigerer Latenz. GGUF-Quantisierung (Q4_K_M) komprimiert das Modell auf ~4GB und ermöglicht Inferenz auf Consumer-GPUs (RTX 3060 12GB oder äquivalent).[23] [35]

Llama 2 70B bietet maximale Qualität für konzeptuelle Keyword-Extraktion, erfordert jedoch substantielle Hardware (40-80GB VRAM für INT4-Quantisierung). Die Methode eignet sich für Batch-Processing mit hohen Qualitätsanforderungen, ist jedoch für Echtzeit-Anwendungen impraktikabel.[38]

Phi-4 (14B) und **SmolLM2 (1.7B)** emergieren als spezialisierte Optionen für ressourcenbeschränkte Umgebungen. Phi-4 erreicht 8/10 in Keyword-Extraktion-Benchmarks bei nur 14B Parametern, während SmolLM2 CPU-Inferenz mit akzeptabler Latenz (<2s pro Dokument) ermöglicht.[25] [37]

Implementierungsmuster mit Ollama

Ollama abstrahiert LLM-Management und bietet eine einheitliche API für lokale Modellausführung. Die Installation und Konfiguration erfolgt in drei Schritten:[39] [40] [41]

```
# Installation
curl -fsSL https://ollama.ai/install.sh | sh

# Modell pullen
ollama pull mistral:7b-instruct

# LLM starten
ollama run mistral:7b-instruct
```

Pattern 1: Strukturierte Prompt-Engineering für deterministische Extraktion:[24] [42] [43]

```
from langchain_community.llms import Ollama
from langchain.prompts import PromptTemplate

llm = Ollama(model="mistral:7b-instruct")

prompt = PromptTemplate(
```

```
    input_variables=["text"],
    template="""Extrahiere die 5 wichtigsten Keywords aus folgendem Text.

Anforderungen:
- Keywords müssen im Text vorkommen
- Fokus auf Nomen und Nominalphrasen
- Ausgabe als kommaseparierte Liste

Text: {text}

Keywords:"""
)

chain = prompt | llm
keywords = chain.invoke({"text": article_text})
```

Pattern 2: KeyBERT + KeyLLM Fusion für optimale Qualität:[2] [21] [22]

```
from keybert import KeyLLM
from keybert.llm import TextGeneration
from sentence_transformers import SentenceTransformer
import ollama

# Embedding-Modell für Kandidatengenerierung
embedding_model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

# Ollama-Generator
def ollama_generator(prompt):
    response = ollama.generate(model='mistral:7b-instruct', prompt=prompt)
    return response['response']

# Prompt-Template
template = """
[INST]Analysiere folgende Keywords und bewerte ihre Relevanz für das Dokument.
Dokument: {document}
Kandidaten: {candidates}

Wähle die 5 relevantesten Keywords.[/INST]
"""

# KeyLLM initialisieren
llm = TextGeneration(ollama_generator, prompt=template)
kw_model = KeyLLM(llm)

# Extraktion mit Embeddings
embeddings = embedding_model.encode([document])
keywords = kw_model.extract_keywords(
    document,
    embeddings=embeddings,
    threshold=0.6
)
```

Pattern 3: Efficient Clustering für Hochvolumen-Szenarien:[21] [2]

```
from sklearn.cluster import DBSCAN
from sentence_transformers import SentenceTransformer

# Dokumente zu Embeddings
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(documents)

# Clustering ähnlicher Dokumente
clustering = DBSCAN(eps=0.3, min_samples=2, metric='cosine')
clusters = clustering.fit_predict(embeddings)

# LLM-Inferenz nur für Cluster-Repräsentanten
representative_docs = {}
for cluster_id in set(clusters):
    if cluster_id == -1: continue  # Noise
    cluster_mask = clusters == cluster_id
    cluster_docs = [doc for doc, mask in zip(documents, cluster_mask) if mask]
    representative_docs[cluster_id] = cluster_docs[^0]

# Keyword-Extraktion
for cluster_id, rep_doc in representative_docs.items():
    keywords = kw_model.extract_keywords(rep_doc)
    # Keywords auf alle Dokumente im Cluster anwenden
    for doc in cluster_docs:
        doc_keywords[doc] = keywords
```

Diese Implementierung reduziert LLM-Calls um 70-90% bei minimal degradierter Qualität (threshold=0.6 → ~3% F1-Verlust).[21]

Performance-Optimierung

Quantisierung komprimiert Modellgewichte von FP32 (32-bit) auf INT4 (4-bit) und reduziert Speicherbedarf um ~87% bei <5% Genauigkeitsverlust. GPTQ-Algorithmus optimiert Quantisierung durch adaptives Rounding:[34]

```
# GGUF Q4_K_M für optimales Präzision-Effizienz-Verhältnis
ollama pull mistral:7b-instruct-q4_K_M
```

Context Window Management limitiert Input auf 512-1024 Tokens für Keyword-Extraktion – längere Kontexte erhöhen Latenz quadratisch ohne proportionale Qualitätsverbesserung. Dokumentsegmentierung mit Overlap (128 Token) für lange Texte:[4] [34]

```
def segment_document(text, max_tokens=512, overlap=128):
    tokens = tokenizer.encode(text)
    segments = []
    for i in range(0, len(tokens), max_tokens - overlap):
        segment = tokens[i:i + max_tokens]
        segments.append(tokenizer.decode(segment))
    return segments
```

Batch Processing amortisiert Model-Loading-Overhead über multiple Dokumente. Ollama unterstützt batch inference nativ:

```
documents = [doc1, doc2, ..., doc_n]
prompts = [prompt_template.format(text=doc) for doc in documents]

# Batch-Inferenz
responses = ollama.generate(
    model='mistral:7b-instruct',
    prompts=prompts,
    options={'num_ctx': 512, 'num_batch': 8}
)
```

Hardware-Anforderungen und Kostenanalyse

Lokale LLM-Inferenz verschiebt Kosten von variablen API-Gebühren zu fixen Hardware-Investitionen. Eine Kosten-Nutzen-Analyse für 1 Million Dokumente (avg. 500 Tokens):

Cloud-LLMs (GPT-4o-mini, $0.15/1M input tokens):
500M Tokens × $0.15 = $75.000 + Netzwerk-Latenz + Vendor-Lock-in

Lokale GPU (NVIDIA RTX 4090, ~$1.800):

- Hardware: $1.800 (einmalig)
- Strom: 450W × 200h × $0.12/kWh = $10.80
- Inferenz: ~4.000 docs/Stunde → 250h Laufzeit
- **Total: ~$1.811 (Break-even bei ~12k Dokumenten)**

Die lokale Ausführung amortisiert sich bei regulären Workloads innerhalb von Wochen und eliminiert langfristige Cloud-Abhängigkeiten.[25]

Evaluationsmetriken und Benchmarking

Robuste Evaluation erfordert präzise Metriken, die sowohl exakte als auch semantische Übereinstimmung erfassen.

Standard-Metriken

Precision@k quantifiziert die Relevanzrate extrahierter Keywords:

$$\text{Precision@k} = \frac{|\text{Extrahierte Keywords} \cap \text{Ground Truth}|}{k}$$

Precision beantwortet: „Von den k extrahierten Keywords, wie viele sind relevant?" Hohe Precision minimiert False Positives, kritisch für Anwendungen wie Dokumentindexierung.[11] [44]

Recall@k misst Coverage relevanter Keywords:

$$\text{Recall@k} = \frac{|\text{Extrahierte Keywords} \cap \text{Ground Truth}|}{|\text{Ground Truth}|}$$

Recall beantwortet: „Von allen relevanten Keywords, wie viele wurden extrahiert?" Hohe Recall-Werte sind essenziell für Information Retrieval, wo das Verpassen wichtiger Begriffe Nutzeranfragen degradiert.[44] [11]

F1@k harmonisiert Precision und Recall:

$$\mathrm{F1@k} = \frac{2 \times \mathrm{Precision@k} \times \mathrm{Recall@k}}{\mathrm{Precision@k} + \mathrm{Recall@k}}$$

F1 balanciert beide Dimensionen und dient als primäre Metrik in akademischen Benchmarks.[18] [45] [28] [44]

Fuzzy Semantic Matching

Exakte String-Übereinstimmung scheitert bei lexikalischen Variationen („Machine Learning" vs. „machine learning") und semantischen Äquivalenzen („KI" vs. „Künstliche Intelligenz"). **Levenshtein Distance** quantifiziert Edit-Distanz zwischen Strings:[6] [45]

$$\mathrm{Levenshtein}(s_1, s_2) = \min(\mathrm{insertions}, \mathrm{deletions}, \mathrm{substitutions})$$

Normalisierte Levenshtein-Ähnlichkeit:

$$\mathrm{Similarity} = 1 - \frac{\mathrm{Levenshtein}(s_1, s_2)}{\max(|s_1|, |s_2|)}$$

Ein Threshold von 0.85 (85% Ähnlichkeit) klassifiziert Near-Matches als korrekt. Fortgeschrittene Frameworks kombinieren Levenshtein mit Embedding-basierten Similarity-Scores (Cosine Similarity > 0.8) für robuste semantische Evaluation.[45]

Benchmark-Datasets

KP20k: 20.000 wissenschaftliche Artikel mit autor-annotierten Keywords. Domäne: Computer Science. Avg. Keywords: 5.5 pro Dokument. State-of-the-art F1@5: 38.7% (SEKE).[18] [20]

Inspec: 2.000 Abstracts aus Informatik- und Elektrotechnik-Journals. Kurze Dokumente (avg. 128 Wörter). Etabliert für short-text benchmarking.[10] [18]

KPTimes/JPTimes: News-Artikel mit journalistisch vergebenen Keywords. Domäne: Aktuelle Ereignisse. Challenges: dynamische Themen, Eigennamen, temporale Relevanz.[4] [18]

SemEval-2010: Multi-domänen-Korpus mit wissenschaftlichen Artikeln. Avg. 146 Wörter/Dokument. LDA erreicht beste Performance aufgrund Topic-Konzentration.[11]

Sprachspezifische Implementierung: Deutsche Texte

Deutsche Keyword-Extraktion erfordert adaptierte Tools und Modelle für morphologische Komplexität (Komposita, Deklinationen) und syntaktische Strukturen.

Sprachmodelle und Tools

Sprachspezifische Embeddings: `paraphrase-multilingual-MiniLM-L12-v2` bietet robuste Performance für deutsche Texte mit 50+ Sprachen. Alternativ: `gbert-base` (monolinguales deutsches BERT) für maximale Präzision.[46]

POS-Tagging: RFTagger mit deutschen Tagsets annotiert Wortklassen und Lemmatisierung. Stanford CoreNLP mit deutschem Modell für dependency parsing.[27] [29]

Stoplisten: Standard NLTK deutsche Stoplisten (232 Wörter) sollten domänenspezifisch erweitert werden – Fachwortschatz eliminiert generische Terme wie „Thema", „Aspekt", „Bereich".

Performance-Charakteristika

Evaluationen auf deutschen Korpora (100.673 Texte von heise.de) zeigen, dass **Topic Models mit LDA** alle anderen Methoden übertreffen, inklusive TextRank und TF-IDF. Der Topic-basierte Ansatz erreicht die höchste Informationsgehalt-Korrelation mit manuell annotierten Keywords. **Recurrent Neural Networks (RNNs)** performen schwach (niedrigste Scores), vermutlich aufgrund insuffizienter Trainingsdaten für morphologisch reiche Sprachen. Bi-LSTM-Modelle mit adequatem Training (90.000 Dokumente) erreichen 76.97% Recall für deutsche CE-Standard-Dokumente, übertreffen RNN (72.99%) und CRF (70.18%).[47] [48]

Pattern-basierte Extraktion mit POS-Sequenzen (z.B. „Adjektiv+Nomen+Präposition+Nomen") identifiziert Termini-Kandidaten, wobei Dependency-Parse-Filterung False Positives reduziert. Die „soft filter" approach verbessert Precision bei Komposita-Kandidaten signifikant.[27]

Praktische Code-Vorlagen

Minimale TF-IDF-Implementierung

```
from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np

# Korpus laden
documents = ["Text Dokument 1", "Text Dokument 2", ...]

# TF-IDF berechnen
vectorizer = TfidfVectorizer(
    max_df=0.85,           # Ignore terms in >85% docs
    stop_words='german',   # Deutsche Stopliste
    ngram_range=(1, 3)     # Uni-, Bi-, Trigrams
)

tfidf_matrix = vectorizer.fit_transform(documents)
feature_names = vectorizer.get_feature_names_out()

# Keywords für ein Dokument extrahieren
doc_index = 0
scores = tfidf_matrix[doc_index].toarray()[^0]
top_indices = scores.argsort()[-10:][::-1]
```

```
keywords = [(feature_names[i], scores[i]) for i in top_indices]

print(keywords)
```

KeyBERT mit Diversifizierung

```
from keybert import KeyBERT
from sentence_transformers import SentenceTransformer

# Deutsches Embedding-Modell
model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
kw_model = KeyBERT(model)

text = """Ihre Textdaten hier..."""

# MMR für diversifizierte Keywords
keywords = kw_model.extract_keywords(
    text,
    keyphrase_ngram_range=(1, 3),  # 1-3 Wörter
    stop_words='german',
    use_mmr=True,                  # Maximal Marginal Relevance
    diversity=0.7,                 # Höher = mehr Diversität
    top_n=10
)

print(keywords)
```

spaCy-basierte Noun-Chunk-Extraktion

```
import spacy
from collections import Counter

# Deutsches spaCy-Modell
nlp = spacy.load("de_core_news_lg")

text = """Ihre Textdaten hier..."""
doc = nlp(text)

# Noun Chunks mit TF-IDF-ähnlichem Filtering
chunks = [chunk.text.lower() for chunk in doc.noun_chunks
          if len(chunk.text) > 3]  # Min. 3 Zeichen

# Frequenz-basiertes Ranking
chunk_freq = Counter(chunks)
top_keywords = chunk_freq.most_common(10)

print(top_keywords)
```

Hybrid-Pipeline: YAKE + KeyBERT

```
import yake
from keybert import KeyBERT

# YAKE für schnelle Kandidatengenerierung
yake_extractor = yake.KeywordExtractor(
    lan="de",
    n=3,                    # Max n-gram
    dedupLim=0.9,
    top=20                  # Top 20 Kandidaten
)

yake_keywords = yake_extractor.extract_keywords(text)
candidates = [kw for kw, score in yake_keywords]

# KeyBERT für semantisches Reranking
kw_model = KeyBERT('paraphrase-multilingual-MiniLM-L12-v2')
final_keywords = kw_model.extract_keywords(
    text,
    candidates=candidates,  # YAKE-Kandidaten
    top_n=10
)

print(final_keywords)
```

Zusammenfassung und Handlungsempfehlungen

Keyword-Extraktion hat sich von einfachen frequenzbasierten Heuristiken zu sophistizierten hybriden Systemen entwickelt, die statistische Methoden, graph-basierte Algorithmen, neuronale Embeddings und LLMs integrieren. Die aktuelle Best Practice (Stand Januar 2026) empfiehlt einen mehrschichtigen Ansatz:

Für Produktionssysteme mit hohem Volumen: Implementieren Sie eine Hybrid-Pipeline mit RAKE/YAKE für initiale Kandidatengenerierung (< 1s Latenz), KeyBERT für semantisches Reranking (2-5s) und optional LLM-basierte Filterung für ambivalente Fälle. Diese Architektur balanciert Effizienz (Durchsatz: 500+ Dokumente/Minute) gegen Präzision (F1: 85-87%).

Für qualitätskritische Anwendungen mit Trainingsdaten: Fine-tunen Sie DeBERTa mit LoRA auf domänenspezifischen Korpora (1.000-5.000 gelabelte Dokumente). Die resultierenden Modelle erreichen SOTA-Performance (F1: 88%) und amortisieren Trainingsaufwand durch konsistent überlegene Extraktion.

Für ressourcenbeschränkte Umgebungen: Nutzen Sie lokale LLMs (Mistral 7B via Ollama) mit efficient clustering. Der Ansatz reduziert Cloud-Kosten um >95% bei minimal degradierter Qualität (3-5% F1-Verlust gegenüber Cloud-LLMs) und gewährleistet Datensouveränität.

Für deutsche Texte: Priorisieren Sie Topic Models (LDA) für längere Dokumente und YAKE für kurze Texte. Integrieren Sie deutsche Stoplisten, nutzen Sie multilingual Embeddings und implementieren Sie Komposita-spezifisches Preprocessing.

Die Konvergenz von Transformer-Modellen, lokaler Inferenz-Infrastruktur (Ollama, GGUF) und hybriden Methoden (TRISUM, MAPEX) demokratisiert hochwertige Keyword-Extraktion für Organisationen jeder Größe. Die strategische Methodenwahl – fundiert auf Datencharakteristika, Ressourcen und Performance-Anforderungen – determiniert den ROI dieser Technologien in praktischen NLP-Pipelines.

⁂

1. https://superlinear.eu/insights/articles/mastering-nlp-a-guide-to-keyword-extraction
2. https://maartengrootendorst.com/blog/keyllm/
3. https://dev.to/talhaahsan/trisum-a-hybrid-graph-based-keyword-extraction-algorithm-1p2c
4. https://arxiv.org/html/2509.18813v2
5. https://www.geeksforgeeks.org/nlp/keyword-extraction-methods-in-nlp/
6. https://www.johnsnowlabs.com/the-experts-guide-to-keyword-extraction-from-texts-with-spark-nlp-and-python/
7. https://ceur-ws.org/Vol-3909/Paper_9.pdf
8. https://towardsdatascience.com/keyword-extraction-a-benchmark-of-7-algorithms-in-python-8a905326d93f/
9. https://www.oreateai.com/blog/common-keyword-extraction-algorithms-in-natural-language-processing-part-9/f10bb2cc82f0e54b78542459f83f975c
10. https://pmc.ncbi.nlm.nih.gov/articles/PMC7512597/
11. https://public.websites.umich.edu/~zmohamed/PDFs/mipr2019.pdf
12. https://github.com/MaartenGr/KeyBERT
13. https://www.oncrawl.com/data-driven-seo/automatically-extract-concepts-keywords-from-text-part-ii-semantic-approach/
14. https://maartengrootendorst.com/blog/keybert/
15. https://towardsdatascience.com/arxiv-keyword-extraction-and-analysis-pipeline-with-keybert-and-taipy-2972e81d9fa4/
16. https://jurnal.polibatam.ac.id/index.php/JAIC/article/download/9971/2915/32859
17. https://arxiv.org/pdf/2312.0909.pdf
18. https://arxiv.org/html/2412.14087v2
19. https://www.thaiscience.info/Journals/Article/ECTI/10997021.pdf
20. https://aclanthology.org/2025.findings-emnlp.764/
21. https://newsletter.maartengrootendorst.com/p/introducing-keyllm-keyword-extraction
22. https://maartengr.github.io/KeyBERT/guides/keyllm.html
23. https://www.youtube.com/watch?v=xF2UJTmRU_Y
24. https://haystack.deepset.ai/cookbook/keyword-extraction
25. https://dev.to/mashrulhaque/stop-paying-openai-free-local-ai-in-net-with-ollama-50k8
26. https://oss.cs.fau.de/2022/10/17/final-thesis-implementing-paket-a-production-ready-ai-enhanced-keyword-extractor/
27. https://elex.link/elex2015/proceedings/eLex_2015_33_Rosiger+etal.pdf

28. https://arxiv.org/pdf/2211.05031.pdf
29. https://projekt.bht-berlin.de/fileadmin/projekt/delfi-wsdq/previews/DELFI2020-preprint__Automatisierte_Extraktion_semantischer_Kompetenzbeschreibungen.pdf
30. https://kavita-ganesan.com/python-keyword-extraction/
31. https://jaketae.github.io/study/keyword-extraction/
32. https://www.geeksforgeeks.org/nlp/keyphrase-extraction-in-nlp/
33. https://pmc.ncbi.nlm.nih.gov/articles/PMC9202614/
34. https://www.nature.com/articles/s41598-025-85715-7
35. https://huggingface.co/docs/transformers/model_doc/mistral
36. https://journals.sagepub.com/doi/10.1177/18758967251392652
37. https://www.reddit.com/r/LocalLLaMA/comments/1h77e1y/smallest_llm_for_extracting_data_using_cpu/
38. https://www.reddit.com/r/LocalLLaMA/comments/18i9hj4/how_do_i_get_local_llms_to_return_keywords/
39. https://apidog.com/blog/ollama-cheatsheet/
40. https://www.youtube.com/watch?v=SqXBB_j2c8k
41. https://itnext.io/information-extraction-with-local-llm-94524c5a1fc6
42. https://www.theprompter.io/p/llms-for-keyword-extraction-a-smarter
43. https://www.youtube.com/watch?v=lGjZ38iqjls
44. https://amitness.com/posts/information-retrieval-evaluation
45. https://aclanthology.org/2025.aisd-main.2/
46. https://www.mlforseo.com/machine-learning-implementation-guides/keyword-research/how-to-do-keyword-clustering-with-keybert/
47. https://www.scitepress.org/Papers/2020/93747/93747.pdf
48. https://www.diva-portal.org/smash/get/diva2:1256706/FULLTEXT01.pdf
49. https://ieeexplore.ieee.org/document/10363189/
50. https://lm-kit.com/blog/introducing-keyword-extraction-api/
51. https://techstrong.ai/aiops/extracting-keywords-from-text-using-natural-language-processing/
52. https://searchatlas.com/blog/nlp-keywords/
53. https://blix.ai/blog/text-analysis-tools
54. https://intrafind.com/en/blog/natural-language-processing-best-practice-en
55. https://lab.abilian.com/Tech/AI & Machine Learning/Keyword Extraction with LLMs/
56. https://thecxlead.com/tools/best-text-analysis-tool/
57. https://www.pageoptimizer.pro/blog/how-to-research-nlp-keywords
58. https://healthark.ai/keyword-extraction-using-regex-tf-idf-and-bert-a-comprehensive-approach/
59. https://towardsdatascience.com/economics-of-hosting-open-source-llms-17b4ec4e7691/
60. https://scrapfly.io/blog/posts/guide-to-local-llm
61. https://www.sciencedirect.com/science/article/pii/S2667305324000954
62. https://thatware.co/text-to-text-transfer-transformer/
63. https://github.com/INESCTEC/yake
64. https://aclanthology.org/2025.aisd-main.2.pdf

65. https://milvus.io/docs/v2.4.x/llama_agents_metadata.md
66. https://arxiv.org/abs/2502.07286
67. https://www.decodingai.com/p/from-noisy-docs-to-fine-tuning-datasets
68. https://mlops.systems/posts/2024-06-15-isafpr-first-finetune.html
69. https://www.reddit.com/r/LocalLLaMA/comments/1bdn1hw/finetuning_a_llm_for_better_text_extraction_and/
70. https://www.sciencedirect.com/science/article/pii/S0306437925001516
71. https://arc.cct.ie/cgi/viewcontent.cgi?article=1000&context=msc_da
72. https://community.openai.com/t/fine-tuning-for-use-of-keyword-lists/551252
73. https://arxiv.org/html/2504.21667v1
74. https://spacy.io/usage/linguistic-features
75. https://ieeexplore.ieee.org/document/10256370/
76. https://www.nature.com/articles/s41598-025-13488-0
77. https://www.aimspress.com/article/doi/10.3934/aci.2022007?viewType=HTML
78. http://ftp.informatik.rwth-aachen.de/Publications/CEUR-WS/Vol-706/poster13.pdf
79. https://www.ijert.org/hybrid-algorithm-to-generate-summary-of-documents-by-extracting-keywords
80. https://www.sciencedirect.com/science/article/abs/pii/S0952197623005225
81. https://github.com/kennethleungty/Keyword-Analysis-with-KeyBERT-and-Taipy
82. https://pypi.org/project/keyword-spacy/
83. https://eranfeit.net/llava-image-recognition-in-python-with-ollama-and-vision-language-models/
84. https://dzone.com/articles/extracting-keywords-from-text
85. https://huspacy.github.io/examples/keyphrase/
86. https://intermediate-python.pythonhumanities.com/notebooks/100-keybert.html
87. https://github.com/wjbmattingly/keyword-spacy
88. https://developers.llamaindex.ai/python/framework/understanding/putting_it_all_together/q_and_a/terms_definitions_tutorial/
89. https://www.kaggle.com/datasets/asadullahcreative/ai-models-benchmark-dataset-2026-latest
90. https://arxiv.org/html/2407.14085v1
91. https://github.com/oeg-upm/spanish-termex
92. https://www.kaggle.com/code/akashmathur2212/complete-guide-to-keywords-phrase-extraction
93. https://aclanthology.org/2024.konvens-main.18/
94. https://www.youtube.com/watch?v=VYBQV2aCAVY
95. https://www.youtube.com/watch?v=U7NjHOVLS5Y
96. https://brightdata.com/blog/web-data/best-data-extraction-tools
97. https://stackoverflow.com/questions/33148775/get-values-of-keywords-from-a-string-and-a-template-in-python
98. https://www.reddit.com/r/LanguageTechnology/comments/p1kg4k/performance_evaluation_of_keywords_extraction/
99. https://scrapfly.io/blog/posts/best-web-scraping-tools

100. https://www.kaggle.com/code/qmarva/keyword-extraction-nlp-project-for-beginners